

Algoritmos e Estruturas de Dados 1

Universidade do Minho

2024/2025

Parque de Estacionamento

Relatório do Projeto

Grupo 8

a104359— Gabriela Sousa e Santos-LEGS

a111488 — Maria Nair da Cunha e Silva Lacerda Ângelo-LEGS

a110943— Mariana Silva Gomes-LEGS

Índice

Introdução.....	2
Estruturas usadas para armazenar dados em memória	3
Ficheiros Usados	5
Principais Opções de Implementação	5
Conclusão.....	9

Introdução

No âmbito da disciplina de AED1, foi atribuído como objeto de estudo e realização um projeto de grupo.

O projeto tem como objetivo o desenvolvimento de uma aplicação em linguagem C.

Para o desenvolvimento da mesma, foi disponibilizado um enunciado do projeto, onde são especificadas as funcionalidades que a aplicação deve ter, bem como o contexto em esta vai ser inserida.

O objetivo deste projeto é desenvolver um sistema de gestão de parque de estacionamento que permita registar a entrada e saída de veículos, gerenciar clientes, e calcular os custos de estacionamento com base no tempo de permanência.

Estruturas usadas para armazenar dados em memória

No projeto, foram utilizadas as seguintes estruturas de dados:

- **TipoVeiculo:** Um enum que define os tipos de veículos (DUAS_RODAS, LIGEIRO, LIGEIRO_GRANDE). Esta enumeração facilita a categorização dos veículos e a implementação específica para cada tipo.

Figura 1 – Estrutura para os tipos de veículos

```
//enum permite associar os tipos a um valor inteiro DUAS_RODAS = 0; LIGEIRO = 1; LIGEIRO_GRANDE = 2
typedef enum { DUAS_RODAS, LIGEIRO, LIGEIRO_GRANDE } TipoVeiculo;
```

- **Lugar:** Uma struct que representa um lugar de estacionamento, contendo informações como id (identificador único para cada lugar), lugar, tipo (tipo de veículo permitido no lugar), ocupado (indica se o lugar está ocupado), matricula (matrícula do veículo estacionado), dataEntrada (data de entrada do veículo) e horaEntrada (Hora de entrada do veículo). Esta estrutura, permite armazenar todos os detalhes necessários para gerir cada lugar de estacionamento.

Figura 2 - Estrutura para lugares de estacionamento

```
typedef struct {
    int id; //É um identificador para cada lugar, isto é, por coordenada (0,0) id =0; (0,1) id=1; (0,2) id=3 etc.
    TipoVeiculo tipo; //tipo de lugar
    bool ocupado; //isto é se estiver ocupado ent 1(verdadeiro) caso contrario 0 (falso)
    char matricula[15]; //string da matricula não pode ter mais do que 15 elementos
    char dataEntrada[11];
    char horaEntrada[6];
} Lugar;
```

- **Parque:** Uma struct que representa o parque de estacionamento, contendo informações sobre o número de pisos, linhas, colunas, preços, e uma matriz de Lugar. Esta estrutura centraliza todas as informações do parque, facilitando a gestão e a consulta dos dados.

Figura 3 - Estrutura do parque de estacionamento

```
typedef struct {
    int numeroPisos;
    int linhas;
    int colunas;
    int lugares2rodas[MAX_PISOS]; //numero total de lugares do tipo 2 rodas por piso
    int lugaresLigeiros[MAX_PISOS]; //numero total de lugares do tipo ligeiro por piso
    int lugaresGrandes[MAX_PISOS]; //numero total de lugares do tipo ligeiro grande por piso
    //preços:
    float min15;
    float min30;
    float hora;
    float horasSeguintes;
    float dia;
    Lugar lugares[MAX_PISOS][MAX_LUGARES];
} Parque;
```

- **Cliente:** Uma struct que representa um cliente, contendo informações como nome, morada, telefone, email, e uma lista de veículos. Esta estrutura permite armazenar todos os detalhes necessários para gerir os clientes do parque.

Figura 4 - Estura que representa um cliente

```
typedef struct {
    int id; //É um identificador para cada lugar, isto é, por coordenada (0,0) id =0; (0,1) id=1; (0,2) id=3 etc.
    TipoVeiculo tipo; //tipo de lugar
    bool ocupado; //isto é se estiver ocupado ent 1(verdadeiro) caso contrario 0 (falso)
    char matricula[15]; //string da matricula não pode ter mais do que 15 elementos
    char dataEntrada[11];
    char horaEntrada[6];
} Lugar;
```

Ficheiros Usados

- **config_parque.csv:** Utilizado para iniciar o estado do parque de estacionamento. Contém informações sobre o número de pisos, linhas, colunas, lugares por tipo de veículo, e preços. Este ficheiro (em Excel) é essencial para configurar o parque de acordo com os critérios desejados.
- **clientes.csv:** Utilizado para armazenar informações dos clientes, incluindo o nome, a morada, o telefone, o email e as matrículas dos veículos. Este ficheiro (em Excel) permite manter um registo atualizado dos clientes e dos seus veículos.

Principais Opções de Implementação

- **Gravação de dados em Ficheiros: (exemplo: clientes.c)** Os dados dos clientes são gravados no ficheiro clientes.csv sempre que um novo cliente é registado ou um cliente existente é atualizado. A função criarFicheiroClientes é responsável por essa tarefa.

Figura 5 - Função criarFicheiroClientes, presente em clientes.c

```
// Função para criar um ficheiro Excel para armazenar os dados dos clientes  
void criarFicheiroClientes(const char *filename) {  
    FILE *file = fopen(filename, "w");
```

- **Estrutura do código:** O código foi dividido em funções específicas para cada tarefa, como inicializarParque, criarFicheiroClientes, entradaVeiculo, saídaVeiculo,... Isso facilita a manutenção e a compreensão do código. As estruturas (structs) foram utilizadas para agrupar dados relacionados, como informações sobre lugares de estacionamento e clientes. Isso melhora a organização dos dados e a eficiência do código.

- **Criação de ficheiros:** O projeto utiliza funções de leitura e escrita de ficheiros (fopen, fclose, fgets, fprintf) para manipular os ficheiros de configuração e de clientes. Isso permite a persistência dos dados entre execuções do programa. Antes de abrir um ficheiro, o código verifica se ele existe e se pode ser aberto corretamente. Isso evita erros de execução e garante que os dados sejam manipulados de forma segura.
- **Validações e verificações:** O código inclui várias verificações para garantir que as entradas do utilizador sejam válidas. Por exemplo, ao solicitar o número de pisos, linhas, colunas, e preços, o programa verifica se os valores estão dentro dos limites permitidos.
O código também verifica se o número de clientes e veículos não excede os limites definidos. Isso evita problemas de alocação de memória e garante que o programa funcione corretamente.
- **Algoritmos de pesquisa e ordenação:** O código inclui funções para pesquisar lugares disponíveis no parque de estacionamento. Por exemplo, a função `lugarMaisProximo` procura o lugar mais próximo disponível para um determinado tipo de veículo.
Para além disso, o código inclui funções para pesquisar clientes pelo nome e verificar se um cliente já está registado. Isso facilita a gestão dos clientes e evita duplicações.

- **Cálculo de Custos:** O código inclui uma função para calcular os custos de estacionamento com base no tempo de permanência. A função calcularCusto utiliza a diferença de tempo entre a entrada e a saída do veículo para calcular o valor a ser cobrado.

Figura 6- Função calcularCusto

```
//Função auxiliar para calcular os preços
float calcularCusto(double diffSegundos) {
    int minutosTotais = diffSegundos / 60;
    int horas = (minutosTotais - 60) / 60;
    int dias = minutosTotais / (24 * 60);

    if (minutosTotais <= 15) {
        return parque.min15;
    } else if (minutosTotais <= 30) {
        return parque.min30;
    } else if (minutosTotais <= 60) {
        return parque.hora;
    } else if (minutosTotais <= 24 * 60) {
        return parque.hora + horas * parque.horasSeguintes;
    } else {
        return dias * parque.dia;
    }
}
```

- **Interação com o utilizador:** O programa apresenta menus interativos para o utilizador, permitindo que ele escolha entre diferentes opções, como registar a entrada de um veículo, registar a saída, ver o estado do parque, ...
O código também inclui mensagens de erro e sucesso para informar o utilizador sobre o estado das operações realizadas. Melhorando assim, a experiência do utilizador e facilitando a identificação de problemas.

Funcionalidade	Implementação (0 a 100%)	Descritivo
1. Definir/Alterar a configuração do parque	100%	Permite definir o número de pisos e de lugares (de cada tipo) em cada um dos pisos. A configuração pode ser editada por ficheiro.
2. Definir o valor do custo do parque	90%	Permite definir o custo do parque por unidade de tempo, mas em caso de alteração de preços requer recriação da configuração.
3. Registrar entrada de veículo e associar a lugar de estacionamento	90%	Regista a matrícula, da data/hora e do lugar atribuído/ocupado por esse veículo no parque de estacionamento. O sistema permite tanto a escolha automática de qual será o lugar atribuído, quanto a seleção pelo utilizador, conforme o sub-menu implementado. A data e a hora da entrada do veículo nem sempre são introduzidas pelo utilizador do software.
4. Registrar a saída do veículo e calcular valor a pagar	100%	Indica a matrícula do veículo que está de saída e a aplicação apresenta o custo total de utilização do parque.
5. Consultar a ocupação do parque	100%	Permite consultar os lugares livres e ocupados para cada piso do parque.
6. Listar lugares ocupados há demasiado tempo	0%	Não implementada. O sistema não lista os lugares ocupados há mais do que um determinado número de dias ou horas.
7. Acrescentar novo cliente	100%	Permite adicionar clientes com dados pessoais e até 5 veículos associados. Atualiza o ficheiro de clientes automaticamente.
8. Editar veículos de clientes	100%	Permite adicionar e remover veículos a clientes existentes.
9. Listar custos por cliente num período tempo	0%	Não implementado. O sistema não permite listar custos de veículos ou clientes em um intervalo de tempo.
10. Armazenamento de dados em ficheiro	100%	Todos os dados da aplicação estão armazenados em ficheiro
11. Cálculo do tempo de estacionamento	100%	Calcula e exhibe o tempo total de estacionamento de um veículo com base na matrícula.
12. Exibição duma lista dos veículos estacionados	100%	Mostra uma lista completa de veículos com matrícula, tipo, data e hora de entrada.
13. Visualização de veículos em lugares específicos	100%	Permite localizar veículos estacionados em lugares específicos.

Conclusão

Para concluir, este projeto de AED1 foi um verdadeiro desafio que nos proporcionou uma compreensão mais profunda da matéria lecionada nas aulas, com destaque para o uso de estruturas de dados, ficheiros e apontadores.

O programa foi testado várias vezes para minimizar erros, e as estratégias complementadas visaram otimizar a gestão de dados e garantir a eficiência no tempo de execução.

Apesar do progresso, identificamos possíveis melhorias, como a implementação de funcionalidades adicionais. Além disso, seria interessante armazenar o estado do parque entre sessões, de modo a permitir a continuidade do trabalho após reiniciar o sistema.

Com este projeto, sentimo-nos mais confiantes e preparados para a realização de trabalhos futuros, pois consideramos que o esforço valeu a pena e temos plena consciência de que esta realização nos trouxe inúmeras vantagens para as nossas aprendizagens e para novos trabalhos universitários e até mesmo trabalhos profissionais que venhamos a ter pela frente.